LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: K.Vinay

Roll No.: A24126552086

Date: 31-08-2026

---

1. TITLE

Development of an Interactive To-Do List Application using HTML, CSS, and DOM Manipulation

2. AIM

To create a functional to-do list web application that allows users to add tasks, mark them as complete, and delete them dynamically using JavaScript Event Listeners and DOM manipulation.

3. OBJECTIVE

The objectives of this experiment are:

- To design a clean user interface for task entry using HTML and CSS.
  \n
- To access DOM elements using getElementById.
  \n
- To attach event listeners to buttons for user interaction.
  \n
- To dynamically create, append, and remove HTML elements using JavaScript.
  \n
- To toggle CSS classes for visual state changes.

4. THEORY

JavaScript allows dynamic updates to a web page by interacting with the Document Object Model (DOM). Elements can be created (document.createElement), modified (innerText, className), added (appendChild), or removed (remove()) in response to user events such as button clicks.

5. ALGORITHM / PROCEDURE

1. Create the UI layout with an input field, an Add button, and a list container.
   \n
2. Style the components using CSS for a clean, centered layout.
   \n
3. Select required elements in JavaScript (input, add, list, message).
   \n
4. Add a click event listener to the Add button.

\n
5. On click, validate the input and dynamically create a div for the new task.
\n
6. Create and attach Complete and Delete buttons to the task item.
\n
7. Attach onclick events to toggle strikethrough and remove elements.
\n
8. Append the new task element to the list container and clear the input field.

6. CODE EXPLANATION

The script uses document.createElement() to dynamically generate a new element for every task entered by the user. An eventListener('click') is attached to the Add button so that the task-generation logic fires appropriately upon user interaction. Once generated, appendChild() is used to add the new task element directly into the DOM container. The visual strikethrough effect for completed tasks is handled seamlessly by calling classList.toggle('done'). Finally, the remove() method is invoked to permanently delete the task from the DOM when the user clicks the Delete button.

7. PROGRAM

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>My To-Do List</title>

    <style>
        body {
    font-family: Arial, sans-serif;
    background-color: #dbeafe;
    text-align: center;
    padding: 40px;
    min-height: 100vh;
}


        .container {
            width: 500px;
            margin: auto;
            background-color: white;
            padding: 25px;
            border-radius: 10px;
            box-shadow: 0 0 10px gray;
        }

        input {
            padding: 10px;
            width: 280px;
            border: 1px solid gray;
            border-radius: 5px;
        }

        button {
            padding: 10px 15px;
            border: none;
            border-radius: 5px;
            cursor: pointer;
            margin-left: 5px;
        }

        #addBtn {
            background-color: green;
            color: white;
        }

        .task {
            display: flex;
            align-items: center;
            justify-content: space-between;
            margin-top: 15px;
            padding: 10px;
            background-color: #eeeeee;
            border-radius: 5px;
        }

        .task span {
            flex: 1;
            text-align: left;
        }
```

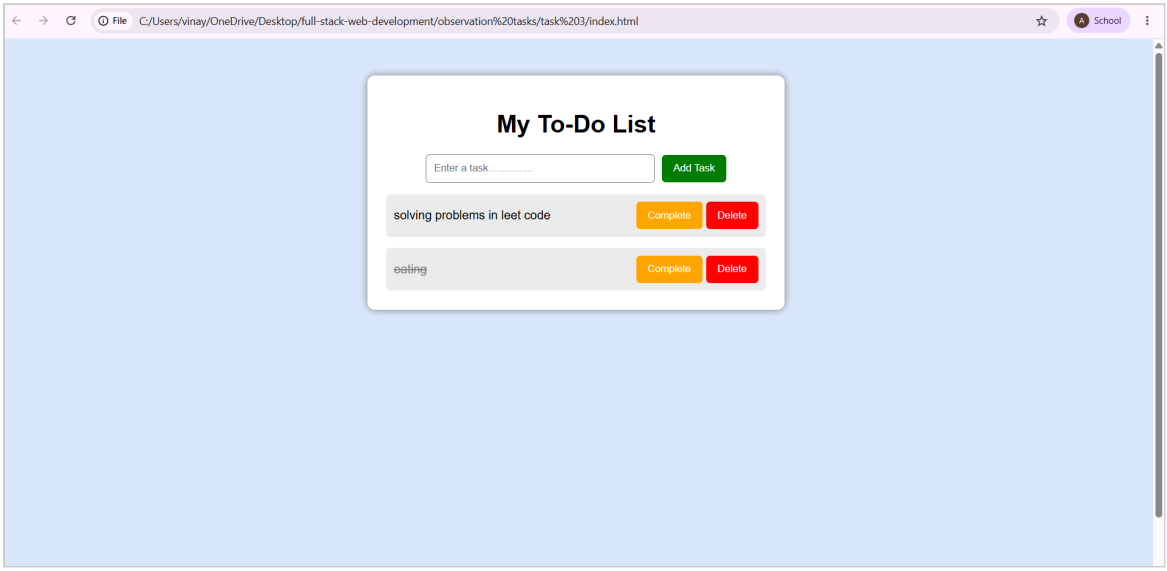
```
        .completeBtn {
            background-color: orange;
            color: white;
        }

        .deleteBtn {
            background-color: red;
            color: white;
        }

        .completed {
            text-decoration: line-through;
            color: gray;
        }
    </style>
</head>

<body>

    <div class="container">

        <h1>My To-Do List</h1>

        <input type="text" id="taskInput" placeholder="Enter a task.................">
        <button id="addBtn">Add Task</button>

        <p id="emptyMessage">No tasks available.</p>

        <div id="taskList"></div>

    </div>

    <script>

        // Select elements from the webpage
        const taskInput = document.getElementById("taskInput");
        const addBtn = document.getElementById("addBtn");
        const taskList = document.getElementById("taskList");
        const emptyMessage = document.getElementById("emptyMessage");

        // Add Task button event
        addBtn.addEventListener("click", function () {

            // Get task text
            const taskText = taskInput.value.trim();

            // Check if input is empty
            if (taskText === "") {
                alert("Please enter a task.");
                return;
            }

            // Create task container
            const task = document.createElement("div");
            task.className = "task";

            // Create task text
```

```
            const text = document.createElement("span");
            text.textContent = taskText;

            // Create Complete button
            const completeBtn = document.createElement("button");
            completeBtn.textContent = "Complete";
            completeBtn.className = "completeBtn";

            // Complete button event
            completeBtn.addEventListener("click", function () {
                text.classList.toggle("completed");
            });

            // Create Delete button
            const deleteBtn = document.createElement("button");
            deleteBtn.textContent = "Delete";
            deleteBtn.className = "deleteBtn";

            // Delete button event
            deleteBtn.addEventListener("click", function () {

                // Remove task
                task.remove();

                // Show message if no tasks remain
                if (taskList.children.length === 0) {
                    emptyMessage.style.display = "block";
                }
            });

            // Add elements to task
            task.appendChild(text);
            task.appendChild(completeBtn);
            task.appendChild(deleteBtn);

            // Add task to task list
            taskList.appendChild(task);

            // Hide empty message
            emptyMessage.style.display = "none";

            // Clear input box
            taskInput.value = "";
        });

    </script>

</body>
</html>
```

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Add Task	Enter task and click Add	Task is appended to the list	**Pass**
TC-02	Complete Task	Click Complete button	Strikethrough style is applied	**Pass**
TC-03	Delete Task	Click Delete button	Task is removed from the DOM	**Pass**

9. OUTPUT



10. RESULT

Thus, an interactive To-Do List application was successfully developed, demonstrating effective DOM manipulation and event handling to manage dynamic task creation, completion, and deletion.